

Jetpack Compose 中核心概念及状态管理与恢复

我们再来详细梳理 Jetpack Compose 中这5个核心概念，用更通俗的语言和贴近实际开发的例子讲解，确保新手也能理解。

1. 无状态组件 (Stateless Composable)

通俗理解：无状态组件就像一个“纯展示的工具”，它的样子完全由外部给的“材料”（参数）决定，自己不保存任何数据。比如一个显示消息的文本框，给它什么文字就显示什么，不会自己偷偷改内容。

核心特点：

- 没有内部状态（不包含 `var` 变量或 `mutableStateOf` 这类状态容器）
- 相同输入参数，一定显示相同UI（可预测性强）
- 易于复用和测试（因为逻辑简单，只依赖输入）

代码示例：

Code block

```
1  // 无状态组件：只接收参数，不保存任何状态
2  @Composable
3  fun ScoreDisplay(score: Int, title: String) {
4      // 整个UI完全由外部传入的 score 和 title 决定
5      Column(
6          modifier = Modifier.padding(16.dp),
7          horizontalAlignment = Alignment.CenterHorizontally
8      ) {
9          Text(text = title, fontSize = 18.sp, color = Color.Gray)
10         Text(text = score.toString(), fontSize = 24.sp, fontWeight =
            FontWeight.Bold)
11     }
12 }
13
14 // 使用方式：传入不同参数，显示不同内容
15 @Composable
16 fun GameScreen() {
17     // 这里的分数是外部状态，传给无状态组件展示
18     ScoreDisplay(score = 80, title = "你的得分") // 显示“你的得分：80”
19     ScoreDisplay(score = 100, title = "最高分") // 显示“最高分：100”
```

为什么要用？ 比如上面的 `ScoreDisplay` 既可以显示当前得分，也可以显示历史最高分，只需要改参数，不用重写组件，复用性拉满。

2. 非结构化状态 (Unstructured State)

通俗理解：非结构化状态就是用普通变量（比如 `var count = 0`）保存的数据，Compose 不知道它的变化，所以就算数据变了，UI 也不会更新。就像你偷偷改了考试分数，但老师没看到，成绩单还是原来的分数。

问题所在：Compose 是“声明式UI”，只有当它能感知到状态变化时，才会重新绘制UI。普通变量的变化，Compose 感知不到，导致“数据和UI不同步”。

代码示例（反面教材）：

Code block

```
1  @Composable
2  fun BrokenCounter() {
3      // 非结构化状态：普通变量，Compose 感知不到变化
4      var clickCount = 0
5
6      Button(
7          onClick = {
8              // 点击时确实会让 clickCount 增加（数据变了）
9              clickCount++
10             println("实际点击次数: $clickCount") // 日志会显示正确次数
11         }
12     ) {
13         // 但UI不会更新，始终显示“点击了 0 次”（UI没变）
14         Text("点击了 $clickCount 次")
15     }
16 }
```

运行结果：点击按钮时，日志会打印 `实际点击次数：1、2、3...`，但按钮上的文字永远是 `点击了 0 次`。

结论：在 Compose 中，永远不要用普通变量保存需要驱动UI变化的状态！

3. 单向数据流 (Unidirectional Data Flow, UDF)

通俗理解：数据像一条单行道，只能按“用户操作 → 改状态 → 刷新UI”的方向流动，不能反向。比如你在计算器上按“+1”（用户操作），计算器内部的数字（状态）加1，然后屏幕（UI）显示新数字，屏幕不会自己偷偷改数字。

核心规则：

- UI 只负责“展示状态”，不直接修改状态
- 状态修改逻辑集中在一个地方（比如父组件或ViewModel）
- 状态是“唯一数据源”，UI 完全由状态决定

代码示例：

Code block

```
1  // 1. 无状态UI组件（只展示，不修改状态）
2  @Composable
3  fun TodoItem(
4      todoText: String,          // 要展示的待办文字（状态）
5      onDeleteClick: () -> Unit // 删除按钮的点击事件（由外部处理）
6  ) {
7      Row(
8          modifier = Modifier.padding(8.dp),
9          horizontalArrangement = Arrangement.SpaceBetween
10     ) {
11         Text(text = todoText) // 只展示状态
12         Button(onClick = onDeleteClick) { // 点击事件交给外部
13             Text("删除")
14         }
15     }
16 }
17
18 // 2. 状态持有者（管理状态和修改逻辑）
19 @Composable
20 fun TodoList() {
21     // 状态：待办列表（用Compose状态容器管理）
22     val todos = remember { mutableStateOf(listOf("学习Compose", "写代码")) }
23
24     // 处理删除逻辑（修改状态的唯一入口）
25     val onDelete = { todoToRemove: String ->
26         todos.value = todos.value.filter { it != todoToRemove }
27     }
28
29     Column {
30         // 遍历状态，为每个待办项创建UI组件
31         todos.value.forEach { todo ->
32             TodoItem(
33                 todoText = todo,
34                 onDeleteClick = { onDelete(todo) } // 传递删除事件
35             )
36         }
37     }
```

数据流向：

用户点击“删除”按钮 → 触发 `onDeleteClick` → 调用 `onDelete` 函数修改 `todos` 状态 → Compose 感知状态变化 → 重新渲染 `TodoList` 和 `TodoItem` → UI 显示更新后的列表。

整个过程数据只按一个方向流动，逻辑清晰，不会出现“不知道谁改了数据”的问题。

4. Compose 的状态管理

通俗理解：状态管理就是用 Compose 提供的“特殊容器”保存数据，让 Compose 能感知数据变化，从而自动刷新UI。就像给数据装了个“报警器”，数据一变就通知 Compose：“快更新UI！”

常用状态容器：

容器	作用	适用场景
<code>mutableStateOf</code>	最基础的状态容器，包装数据（如数字、字符串、对象）	所有需要驱动UI变化的状态
<code>remember</code>	配合 <code>mutableStateOf</code> 使用，确保组件“重新绘制”时状态不丢失	临时状态（如页面内的计数器）
<code>rememberSaveable</code>	在 <code>remember</code> 基础上，支持屏幕旋转、Activity重建时保存状态	需要跨配置变化的状态

代码示例（基础用法）：

Code block

```
1  @Composable
2  fun WorkingCounter() {
3      // 正确的状态管理：remember + mutableStateOf
4      // remember: 确保组件重新绘制时，count 不会被重置为0
5      // mutableStateOf: 包装数据，让Compose能感知变化
6      val count = remember { mutableStateOf(0) }
7
8      Button(
9          onClick = {
10             // 修改状态（必须通过 .value 访问）
11             count.value++
12         }
13     ) {
```

```
14          // 状态变化时，Text会自动刷新
15          Text("点击了 ${count.value} 次")
16      }
17  }
```

代码示例（处理屏幕旋转）：

```
Code block

1  @Composable
2  fun RotationSafeCounter() {
3      // 用 rememberSaveable 替代 remember，旋转屏幕后状态不丢失
4      val count = rememberSaveable { mutableStateOf(0) }
5
6      Column(modifier = Modifier.padding(16.dp)) {
7          Button(onClick = { count.value++ }) {
8              Text("点击了 ${count.value} 次")
9          }
10         Text("旋转屏幕，次数不会重置哦～")
11     }
12 }
```

为什么需要这些容器？

- 没有 `mutableStateOf`：Compose 不知道数据变了，不刷新UI（就是前面说的非结构化状态问题）。
- 没有 `remember`：组件每次重新绘制（重组）时，状态会被重置（比如计数器每次点完都变回0）。
- 没有 `rememberSaveable`：屏幕旋转时，状态会丢失（比如横屏切竖屏，计数器变回0）。

5. Compose 的状态恢复

通俗理解：状态恢复就是让应用在“意外情况”（比如屏幕旋转、应用被关闭后重启）后，能恢复之前的状态。比如你写作文时突然停电，重启电脑后还能看到之前写的内容。

两种常见场景及解决方案：

场景	解决方案	原理
屏幕旋转、Activity重建	使用 <code>rememberSaveable</code>	数据会被保存到 <code>Bundle</code> 中，重建后恢复
应用被杀死后重启	配合持久化存储（如 <code>DataStore</code> 、	数据保存到文件，重启后读取

代码示例1：屏幕旋转恢复（用 rememberSaveable）：

Code block

```

1  @Composable
2  fun FormScreen() {
3      // 保存输入框内容，旋转屏幕不丢失
4      val username = rememberSaveable { mutableStateOf("") }
5      val password = rememberSaveable { mutableStateOf("") }
6
7      Column(modifier = Modifier.padding(16.dp)) {
8          TextField(
9              value = username.value,
10             onChange = { username.value = it },
11             label = { Text("用户名") }
12         )
13         TextField(
14             value = password.value,
15             onChange = { password.value = it },
16             label = { Text("密码") },
17             visualTransformation = PasswordVisualTransformation()
18         )
19     }
20 }

```

效果：输入一半用户名/密码时旋转屏幕，内容不会丢失。**代码示例2：应用重启恢复（配合DataStore）：**

Code block

```

1  // 1. 定义DataStore（持久化存储）
2  val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name =
    "user_prefs")
3
4  // 2. 状态恢复逻辑
5  @Composable
6  fun PersistentFormScreen(context: Context) {
7      // 从DataStore读取保存的用户名（初始状态）
8      val savedUsername = context.dataStore.data
9          .map { prefs -> prefs[stringPreferencesKey("username")] ?: "" }
10         .collectAsState(initial = "")
11
12         // 用rememberSaveable保存当前输入（处理旋转）
13         val username = rememberSaveable { mutableStateOf(savedUsername.value) }

```

```

14
15 // 输入变化时，保存到DataStore（确保应用重启后能恢复）
16 LaunchedEffect(username.value) {
17     context.dataStore.edit { prefs ->
18         prefs[stringPreferencesKey("username")] = username.value
19     }
20 }
21
22 TextField(
23     value = username.value,
24     onValueChange = { username.value = it },
25     label = { Text("用户名（重启应用也能恢复）") }
26 )
27 }

```

效果：输入用户名后关闭应用，重新打开，之前输入的内容会被恢复。

总结

1. **无状态组件：**纯展示，靠参数决定UI，复用性强。
2. **非结构化状态：**普通变量，无法驱动UI更新，坚决不用。
3. **单向数据流：**用户操作→改状态→刷新UI，数据流向清晰。
4. **状态管理：**用 `remember + mutableStateOf` 管理临时状态，`rememberSaveable` 处理旋转。
5. **状态恢复：**旋转用 `rememberSaveable`，重启用持久化存储（如DataStore）。

掌握这些概念，就能写出状态清晰、行为可预测的 Compose 代码了！

（注：文档部分内容可能由 AI 生成）